

1) Mathematical Model

1.1) Projective Plane

Perspective Projection

Since from 3D to 2D some information is lost, the perspective is non-linear. The coordinates get divided by z , so that farther objects appear smaller.

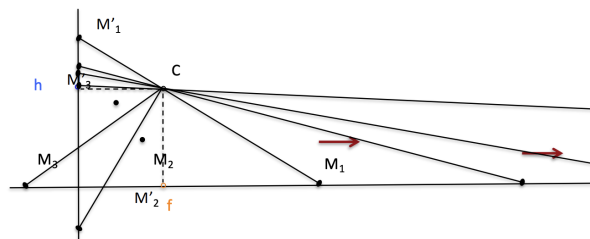
The following equations hold:

$$\frac{f}{z} = \frac{-u}{x} = \frac{-v}{y} \rightarrow \begin{cases} u = -f \frac{x}{z} \\ v = -f \frac{y}{z} \end{cases}$$

Which is a non-linear operation

This allows to represent all 3D points, besides two:

- point inside focal plane (f)
- point on horizontal line (h)



Example

Homogeneous Coordinates

Consider two lines

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a & b \\ a' & b' \end{bmatrix}^{-1} \begin{bmatrix} -c \\ -c' \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \end{bmatrix} = \frac{1}{\det \begin{bmatrix} a & b \\ a' & b' \end{bmatrix}} \begin{bmatrix} bc' - b'c \\ a'c - ac' \end{bmatrix}$$

From here they intersect at

$$x = \frac{bc' - b'c}{ab' - a'b} = \frac{u}{w} \quad y = \frac{a'c - ac'}{ab' - a'b} = \frac{v}{w}$$

where the point (u, v, w) is the point in homogeneous coordinates.

In particular:

- $w = 0$ the point lies at infinity (all of $(u, v, 0)$ form line l_∞)
- $w \neq 0$ point lies in euclidean space
- if (u, v, w) solves the system, then also $\lambda(u, v, w)$ does. This means a single Cartesian point can be represented by infinitely many proportional homogeneous triplets.

Projective Plane

The projective plane $\mathbb{P}^2$ is the union of the euclidean plane $\mathbb{R}^2$ and the line at infinity l_∞ .

Cartesian Plane

Given two different points there exists only one line that contains them

Projective Plane

Given two different points there exists only one line that contains them

Cartesian Plane	Projective Plane
There exists only one line with a given direction and containing a certain point P	Two different lines have only one common point.
Two different lines have either a common point (incident) or the same direction (parallel)	

A line in with homogeneous coordinates is expressed in the following way:

$$ax + by + c = 0 \rightarrow au + bv + cw = 0$$

where $X = (u, v, w)$ is a point and the line coefficients $Y = (a, b, c)$ then the line can be expressed via a dot product $X^T Y = 0$.

Therefore the projective plane contains:

- the euclidean plane $\mathbb{R}^2$; that is, all lines of form (a, b, c) where at most one between a, b is 0.
- the line at infinity l_∞ ; that is, the line of form $(0, 0, c) = \lambda(0, 0, 1)$, $c = \lambda \neq 0$. Here the points are of form $(u, v, 0)$ since $cw = 0 \rightarrow w = 0$. These specify a direction

The projective plane $\mathbb{P}^2$ is made of point triplets $(u, v, w) \neq (0, 0, 0)$ given the equivalence equation $(u, v, w) \sim \lambda(u, v, w) \forall \lambda \neq 0$.

This is like representing points as intersection of lines. In fact consider

$$L_1 = (a, b, c), L_2 = (a', b', c') \rightarrow (u, v, w) = L_1 \times L_2$$

Consider two parallel lines $L_1 = (a, b, c), L_2 = (a, b, c')$ their intersection corresponds in a point at infinity:

$$L_1 \times L_2 = (2b, -2a, 0) = (-b, a, 0)$$

Example for better understanding:

An euclidean point $(2, 4)$ becomes subset of points point $\lambda(2, 4, 1)$ in homogeneous coordinates. This corresponds to a line in $\mathbb{R}^3$ going through the origin and containing all the points $\lambda(2, 4, 1)$. In fact the equation of the line is

$$l(\lambda) = \begin{cases} u = 2\lambda \\ v = 2\lambda \\ w = \lambda \end{cases} = \lambda \cdot \begin{bmatrix} 2 \\ 4 \\ 1 \end{bmatrix}.$$

In general terms the point (x, y) is the homogeneous line $\lambda[x, y, 1]^T = (u, v, w), w \neq 0$

As a recap:

- The homogeneous origin is not defined
- All euclidean points cross the plane $z = 1$ (shifted xy plane) in homogeneous coordinates
- A direction (d_x, d_y) corresponds to the homogeneous point $(d_x, d_y, 0)$
- The origin corresponds to the the z-axis line
- Non euclidean points (points at infinite, directions) never cross the plane $z = 1$.

1.2) Transformations

Affine and Projective Transformations

Definition 3.1 (Projective Transformations).

Projective transform or projectivity is a linear application in homogeneous coordinates

$$f : \mathbb{P}^n \rightarrow \mathbb{P}^n \quad f : x \rightarrow Hx$$

where H is a non singular $n + 1 \times n + 1$ matrix.

- They keep collinearity
- Form a group ($f \in \mathcal{G}_P$)
- H and λH are the same

Definition 3.2 (Affine Transformations).

Affine transform or affinity is a projectivity that maps real points in real points and ideal points in ideal points.

An affinity must be equal to

$$H = \begin{bmatrix} A_{n \times n} & b \\ 0 & 1 \end{bmatrix}$$

- Keeps parallelism

Definition 3.3 (Similarity Transformations).

A **similarity** is represented by

$$H = \begin{bmatrix} sR_{n \times n} & b \\ 0 & 1 \end{bmatrix}$$

with s scalar and R orthogonal matrix

- Keeps angles

With $s = 1$ it is called **euclidean transformation** and keeps distances and lengths.

1.3) Camera Model

Camera Model with Homogeneous Coordinates

In homogeneous coordinates the world-camera relation becomes

$$M = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad m = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

and then

$$Zm = \begin{bmatrix} -fx \\ -fy \\ z \end{bmatrix} = \underbrace{\begin{bmatrix} -f & 0 & 0 & 0 \\ 0 & -f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_P \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = PM$$

and so with respect to scale these are equal

$$m \approx PM$$

- Case $P = [I|0]$:
 - Z is the optical axis
 - Principal Point is the origin of image plane
- General case:
 - No common axis
 - Size and shape of CCD wrt to principal point

Let u_0, v_0 be the coordinates of the principal point, θ the orientation of the sensor wrt to CCD and k_u, k_v the pixel size, then the camera matrix becomes

$$P = \left[\begin{array}{ccc|c} -fk_u & -fk_u \cot \theta & u_0 & 0 \\ 0 & \frac{-fk_v}{\sin \theta} & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{array} \right]$$

Define the **intrinsinc parameter matrix**

$$K = \begin{bmatrix} -fk_u & 0 & u_0 \\ 0 & -fk_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

then $P = K[I_3|0]$

Camera Motion

Now suppose the camera coordinates are not aligned with the world coordinates, then the camera coordinates M_c become:

$$M_c = GM = \begin{bmatrix} R & t \\ 0_{3 \times 1} & 1 \end{bmatrix}$$

so in general we can do the following:

$$m \approx PM_c = K[I_3|0]GM$$

and the **general camera matrix is**

$$P = K[R|t]$$

the coordinates can be normalized as $p = K^{-1}m$
in these coordinates the projection matrix is $P[I|0]$

Principal Point and Center of Projection

A generic camera matrix P can be rewritten according to its rows as:

$$P = \begin{bmatrix} p_1^T \\ p_2^T \\ p_3^T \end{bmatrix} \rightarrow m \approx PM = \begin{bmatrix} p_1^T M \\ p_2^T M \\ p_3^T M \end{bmatrix}$$

so the coordinates of a pixel in the plane becomes (cartesian coords):

$$\begin{cases} u = \frac{p_1^T M}{p_3^T M} \\ v = \frac{p_2^T M}{p_3^T M} \end{cases}$$

Notice that the focal plane contains all the points at infinite, so

$$p_3^T M = 0$$

The points on axis $u = 0$ and $v = 0$ are the projection of 3D points that satisfy $p_1^T M = p_2^T M = 0$ and therefore the COP C becomes $pC = 0$.

In cartesian coordinates the cop becomes

$$\tilde{C} = -Q^{-1}q$$

since by writing $P = [Q|q]$ and $C = [\tilde{C}, 1]^T$ this holds $PC = Q\tilde{C} + q = 0$

Optical Ray

Given a point m in the imaging plane, a **ray** is the line that contains COP C and m , that is, the set of points M such that:

$$\{M : m \approx PM\}$$

Therefore by definition one point is C and another is the ideal point $[Q^{-1}m, 0]^T$ since $P[Q^{-1}m, 0]^T = [Q|q][Q^{-1}m, 0]^T = QQ^{-1}m + q0 = m$

So the ray is obtained as

$$M = C + \lambda \begin{bmatrix} Q^{-1}m \\ 0 \end{bmatrix}, \lambda \in \mathbb{R} \cup \{\infty\}$$

where λ is the variable of the ray that selects the point on the line based on the distance from C .

Example 3.4.

Let Q be the COP and P a pixel in the imaging plane. The optical ray can be written as

$$M = Q + tV, t \in \mathbb{R} \cup \{\infty\}$$

where V is the PQ direction normalized: $PQ = P - Q \rightarrow V = \frac{PQ}{\|PQ\|}$

Now suppose I have a surface with normal n at point S . The intersection between the ray and the surface is at distance (if they intersect)

$$t' = \frac{(S - Q)n}{nV}$$

at point

$$P = Q + t'V$$

and the intersection happens at distance $\|P - S\|$ from S .

2) Illumination

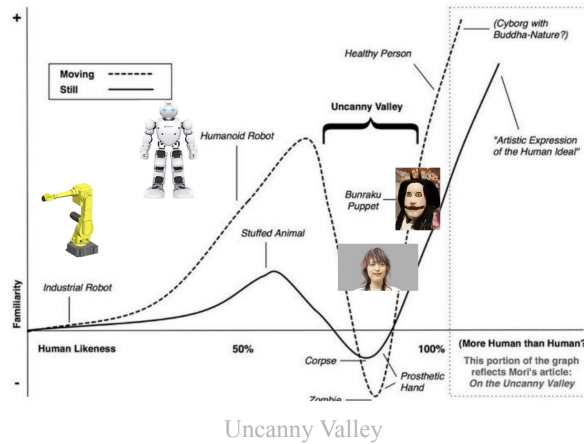
z-buffer vs raster vs ray tracing
todo

3) Animation

3D animation is the spiritual successor of stop motion animation. Some key frames are specified with some key poses and the remaining frames are interpolated.

Uncanny valley: human characters result disturbing

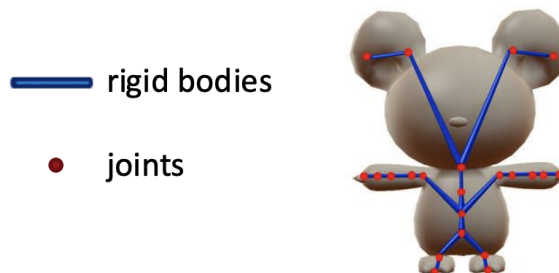
However, non realistic humanoid characters have a better response!



Uncanny Valley

Articulated Characters

We generate a skeleton for the 3d mesh. Mesh is then modified and connected to the elements of the skeleton via a skinning operation.



Example

There are 4 frequent joints:

- Hinge: rotation along 1 axis (1 DoF)
- Saddle: back and forth + up and down (2 DoF)
- Ball-and-socket: rotation along all 3 axes (3 DoF)
- Sliding: translation on a plane (2 DoF)

But a joint can in general support up to 6 DoF

Animating them via key poses is not trivial for complex motions: we resort to **puppeteering** (motion capture).

However real life movements do not work precisely for animation (12 rules of animation) and some very complex elements (hair, water, physics) are simulated.

4) Homography Estimation

3D objects need to be localized in the image.

Definition 3.5 (Homography).

Homography is the function that maps points of plane Π on the image plane

$$\Pi : Z = 0$$

that is, it maps a plane into another plane

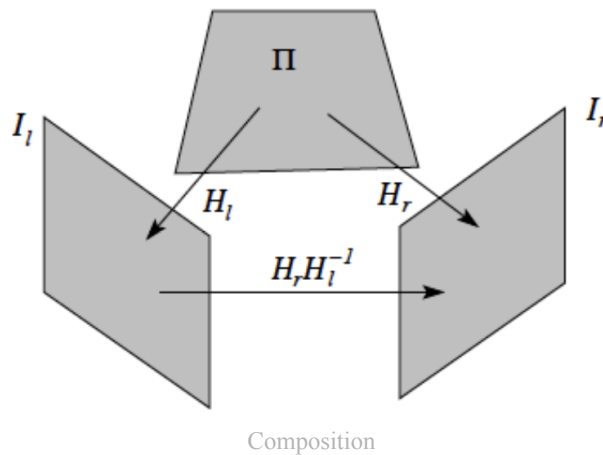
$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} p_{11} & p_{12} & p_{14} \\ p_{21} & p_{22} & p_{24} \\ p_{31} & p_{32} & p_{34} \end{bmatrix}}_H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

that is:

$$m' = H_{\Pi} m \quad m \in \Pi$$

where H is a 3×3 singular matrix defined wrt scale factor (8 DoF).

Homography can also be composed:



Since $m'_i \approx H m_i$ we have that

$$m'_i \times H m_i \approx m'_i \times m'_i = 0$$

then

$$[m'_i]_{\times} H m_i = 0 \rightarrow \text{vec}([m'_i]_{\times} H m_i) = 0 \rightarrow (m_i^T \otimes [m'_i]_{\times}) \text{vec}(H) = 0$$

call $A = m_i^T \otimes [m'_i]_{\times}$ with rank 2 and thus the sizes

$$A_{2n \times 9} \text{vec}(H)_{9 \times 1} = 0$$

Only 4 points are needed for the homography

BASCI LINEAR ALGEBRA RECAP

Scalar product:

$$a^T \cdot b = \langle a, b \rangle = \sum_{i=1}^n a_i b_i = b^T \cdot a = |a||b| \cos \theta \rightarrow \theta = \frac{\langle a, b \rangle}{|a||b|}$$

Outer (vector) product:

$$c = a \times b = |a||b| \sin \theta (u_a \times u_b)$$

let $a = [a_1, a_2, a_3]^T$ similar for b . Then:

$$c = a \times b = \begin{bmatrix} \hat{x} & \hat{y} & \hat{z} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{bmatrix} = \begin{bmatrix} \det \begin{pmatrix} a_2 & a_3 \\ b_2 & b_3 \end{pmatrix} \\ -\det \begin{pmatrix} a_1 & a_3 \\ b_1 & b_3 \end{pmatrix} \\ \det \begin{pmatrix} a_1 & a_2 \\ b_1 & b_2 \end{pmatrix} \end{bmatrix} = \begin{bmatrix} (a_2 b_3 - a_3 b_2) \\ -(a_1 b_3 - a_3 b_1) \\ (a_1 b_2 - a_2 b_1) \end{bmatrix}$$

we can also define the external product matrix:

$$[a]_{\times} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \rightarrow [a]_{\times} b = a \times b$$

moreover:

$$\det(a \times b \times c) = a^T(b \times c) = (a \times b^T)c$$

then also:

$$[A^{-1}u]_{\times} = A^T[u]_{\times}A$$

Norm of a matrix:

- Frobenius: $\|A\| = \sqrt{\sum_{i,j} a_{i,j}^2}$
- Euclidean: $\|A\| = \frac{\|Au\|}{\|u\|}$ with $\forall u : \|u\| \leq 1$

Moore Penrose pseudoinverse:

Singular Value Decomposition (SVD)

Kronecker Product:

$$[A_{m \times n} \otimes B_{p \times q}]_{mp \times nq} = \begin{bmatrix} a_{1,1}B & a_{1,2}B & \dots & a_{1,n}B \\ a_{2,1}B & a_{2,2}B & \dots & a_{2,n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1}B & a_{m,2}B & \dots & a_{m,n}B \end{bmatrix}$$

moreover: $\text{rank}(A \otimes B) = \text{rank}(A)\text{rank}(B)$

Vec operator:

$$\text{vec}(A) = [a_{1,1}, \dots, a_{m,1}, a_{1,2}, \dots, a_{1,n}, \dots, a_{m,n}]^T$$

moreover:

$$\text{vec}(ABC) = (C^T \otimes A)\text{vec}(B)$$

4.2) Epipolar Geometry

5) Stereo Systems

Stereo system uses two views of the same scene to estimate depth. This is done via the disparity between points in a scene.

This chapter will follow this structure:

- Understand the basic case (simple triangulation)
- Understand the general case and how this tells us that conjugate points are on the same epipolar line
- Rectify the images so that the generic case becomes the simple triangulation case
- Given m find the matching point m'

Simple Triangulation

Let two cameras be parallel and aligned (image plane corresponds), then by knowing:

- b distance between cameras. If unknown the distance is a scaling factor
- f focal distance
- $(u - u')$ disparity between two points

The distance z is computed by

$$z = \frac{bf}{u - u'}$$

In this case the camera matrices are

$$P = K[I|0] \quad P' = K[I|(-b, 0, 0)^T]$$

General Case

Suppose we know m but not m' , it is possible to find m' via the epipolar line. This is because the ray of camera C to point M is seen by camera C' as a line (epipolar line).

Call F the 3×3 **fundamental matrix** that encapsulates info about epipolar geometry

$$F = [e']_{\times} Q' Q^{-1}$$

such that the Lounget-Higgins equation becomes:

$$m'^T F m = 0$$

The epipolar line is $l' = Fm$ so the equation tells us that m' lies on l' .

Now suppose that the matching points are known:

In the generic case a stereo system is not in the ideal condition. The generic cameras are

$$P = \begin{bmatrix} p_1^T \\ p_2^T \\ p_3^T \end{bmatrix} \quad P' = \begin{bmatrix} p_1'^T \\ p_2'^T \\ p_3'^T \end{bmatrix}$$

Knowing the two matching points, the point in 3d space M is found by solving this equation

$$\begin{bmatrix} p_1^T M - u p_3^T M \\ p_2^T M - v p_3^T M \\ p_1'^T M - u' p_3'^T M \\ p_2'^T M - v' p_3'^T M \end{bmatrix} = A M = 0_{4 \times 1}$$

With a SVD decomposition of A ($A = U \Sigma V^T$) and by taking the last eigenvector (last vector of V) we get the solution. This can be generalized to N cameras.

Since in real case scenarios due to noise the points don't lie on the epipolar line but slightly diverge, then SVD finds the algebraic minimum, to minimize the geometric cost we can use (find M that minimizes)

$$\epsilon(M) = \left\| \begin{bmatrix} u \\ v \end{bmatrix} - \begin{bmatrix} \frac{p_1^T M}{p_3^T M} \\ \frac{p_2^T M}{p_3^T M} \end{bmatrix} \right\|^2 + \left\| \begin{bmatrix} u' \\ v' \end{bmatrix} - \begin{bmatrix} \frac{p_1'^T M}{p_3'^T M} \\ \frac{p_2'^T M}{p_3'^T M} \end{bmatrix} \right\|^2 \rightarrow M = \arg \min_M \epsilon(M)$$

This however is harder to compute.

Proof of formula.

Let there be 2 cameras with matrices:

$$P = \begin{bmatrix} p_1^T \\ p_2^T \\ p_3^T \end{bmatrix} \quad P' = \begin{bmatrix} p_1'^T \\ p_2'^T \\ p_3'^T \end{bmatrix}$$

where at the i -th row we have the vector p_i^T

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = qPM \rightarrow \begin{cases} u = p_1^T M \\ v = p_2^T M \\ 1 = p_3^T M \end{cases} \rightarrow \begin{cases} u(p_3^T M) = p_1^T M \\ v(p_3^T M) = p_2^T M \end{cases} \rightarrow \begin{cases} p_1^T M - up_3^T M = 0 \\ p_2^T M - vp_3^T M = 0 \end{cases}$$

and the final system of linear equations in matrix form becomes (for each camera)

$$\begin{bmatrix} p_1^T M - up_3^T M \\ p_2^T M - vp_3^T M \end{bmatrix} = 0_{2 \times 1}$$

combining both cameras we obtain

$$\begin{bmatrix} p_1^T M - up_3^T M \\ p_2^T M - vp_3^T M \\ p_1'^T M - u'p_3'^T M \\ p_2'^T M - v'p_3'^T M \end{bmatrix} = AM = 0_{4 \times 1}$$

With a SVD ($A = U\Sigma V^T$) decomposition of A and by taking the last eigenvector (last vector of V) we get the solution. This can be generalized to N cameras. □

Proof that in the simple case the matching points are on the epipolar lines:

Let

$$P = K[I|0] \quad P' = K[I|t = (-b, 0, 0)^T]$$

Find $[t]_{\times}$:

$$[t]_{\times} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & b \\ 0 & -b & 0 \end{bmatrix}$$

Since there is no rotation, $Q'Q^{-1} = I$ and the fundamental matrix is $F = [t]_{\times}I = [t]_{\times}$, then the epipolar line is:

$$l' = Fm = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & b \\ 0 & -b & 0 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ b \\ -bv \end{bmatrix}$$

The Lounget-Higgins equation now is:

- -

$$m'^T F m = [u' \quad v' \quad 1] \begin{bmatrix} 0 \\ b \\ -bv \end{bmatrix} = (u' \cdot 0) + (v' \cdot b) + (1 \cdot -bv) = b(v' - v) = 0 \xrightarrow{b \neq 0} v' = v$$

they are on the same epipolar line (horizontal) $m = [u, v, 1]$, $m' = [u' = u + d, v', 1]$

□

In the general case we have

$$P = K [I|0] \quad P' = K' [R|t]$$

assume that $K = K' = I$. The optical ray of the first image and going to point M is:

$$M = \lambda \begin{bmatrix} m \\ 1 \end{bmatrix} = \lambda \hat{m}$$

These points are projected onto the second camera

$$q' m' = \lambda R m + t$$

Multiply by t to remove scalars (recall $t \times t = 0$) and multiply by m' to remove left term (recall $t \times m' \perp m'$)

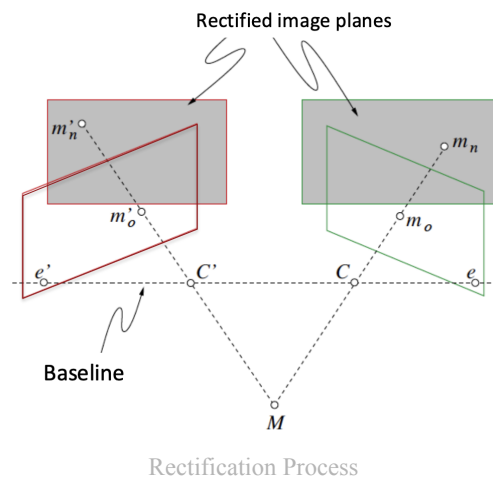
$$t \times (q' m') = t \times (\lambda R m + t) \rightarrow q' (t \times m') = \lambda (t \times R m) \rightarrow m'^T (t \times R m) = 0$$

where $F = t \times R$ is the fundamental matrix

□

Image Rectification

Image rectification is the process of transforming (using homographies) the image so that the focal planes are the same (epipoles at infinite and vertical coordinates of conjugate points are the same). This allows us to find the distance using the simple triangulation.



This is done by rotating around C (or C'). This needs to be computed once, then if the cameras are not moved the rotation matrix is the same.

Conjugate Points Matching

Since the cameras are differently orientated, some matches can be wrong for various reasons:

- **False matches:** given by sift
- **Occlusion:** due to parallax some points can be occluded in one image and be visible in the other. *Their identification is important*
- **Radiometric Distortion:** different lighting conditions based on different angles
- **Perspective Distortion:** change of geometry based on different angles

The constraints we are looking for are:

- **Similarity:** similar pixels in both images
- **Epipolar Geometry:** the correspondence is only on the epipolar line
- **Smoothness:** close pixels have approx same disparity if on same region
- **Uniqueness:** no double matches
- **Ordering:** position depends on camera itself

There are two different classes of algorithms (methods)

- **Local Methods:** uses local characteristic that affects small regions at a time
- **Global Methods:** constraint on whole scan line or entire image

LOCAL METHOD: BLOCK MATCHING

Block matching is a local method that can be summarized by the following passages.

- Around each pixel $m = (u, v)$, and its conjugate candidate $m' = (u + d, v)$ it selects a block of $(2n + 1) \times (2m + 1)$ pixels
- Compute a coupling metric between the block around m and m'
- Operation repeated for different disparity values $d \in [d_{\min}, d_{\max}]$
- Find $d_0(u, v)$ that minimizes/maximizes the metric

Various metrics can be used based on:

- correlation (NCC, ZNCC) (maximize)
- intensity difference (SSD, SAD) (minimize)
- rank operators (census) (minimize)

These metrics are for dense estimation, find matching point for all pixels (in theory)

Small window: not reliable disparity, only reliable if signal is highly variable in the window

Large window: noisy measurements

A solution is to use multi resolution windows or fixed windows.

SUM OF SQUARED DIFFERENCE (SSD):

Minimize

$$SSD(u, v, d) = \sum_{k,l} (I_1(u + k, v + l) - I_2(u + k + d, v + l))^2 \quad k \in [-n, n] \quad l \in [-m, m]$$

Then minimize it

$$d_0(u, v) = \arg \min_{d \in [-d_{\max}, d_{\max}]} SSD(u, v, d)$$

SUM OF ABSOLUTE DIFFERENCES (SAD)

Minimize

$$SAD(u, v, d) = \sum_{k,l} |I_1(u + k, v + l) - I_2(u + k + d, v + l)| \quad k \in [-n, n] \quad l \in [-m, m]$$

And again minimize it

This is better than SAD since:

- is less sensitive to noise
- requires well limited complexity
- better functions can be defined such as Cauchy Functions

larger block size smooths the output.

NORMALIZED CROSS CORRELATION (NCC)

Maximize

$$NCC(u, v, d) = \frac{1}{(2n+1)(2m+1)} \sum_{k,l} \frac{I_1(u+k, v+l)I_2(u+k+d, v+l)}{\sqrt{M_1(u, v)}\sqrt{M_2(u+d, v)}}$$

where M is the mean intensity of the block

$$M = \frac{1}{(2n+1)(2m+1)} \sum_{k,l} I(u+k, v+l)^2$$

ZERO-MEAN NORMALIZED CROSS CORRELATION (ZNCC)

Maximize

This is invariant to illumination changes as it normalizes with the window average (M) and variance (σ)

$$ZNCC(u, v, d) = \frac{1}{(2n+1)(2m+1)} \cdot \sum_{k,l} \frac{(I_1(u+k, v+l) - M_1(u, v))(I_2(u+k+d, v+l) - M_2(u+d, v))}{\sigma_1(u, v)\sigma_2(u+d, v)}$$

with

$$\sigma(u+v) = \frac{1}{(2n+1)(2m+1)} \sum_{k,l} (I(u+k, v+l) - M(u, v))^2$$

CENSUS TRANSFORM

Minimize

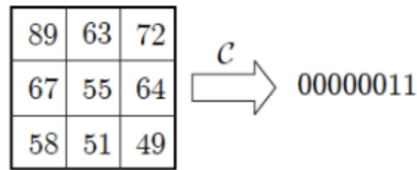
This transform the window in a binary string and then calculates the humming distance between windows

The function that transforms two pixels into a bit is:

$$\varepsilon(I, p, p') = \begin{cases} 1 & I(p) > I(p') \\ 0 & \text{otherwise} \end{cases}$$

And the block is found by concatenating (S is the block)

$$C[I(p)] = \bigodot_{p' \in S} \varepsilon(I, p, p')$$



Example

The **Sum of Census Hamming Distances (SCH)** is

$$SCH(u, v, d) = \sum_{k,l} C[I_1(u+k, v+l) \ominus C[I_2(u+k+d, v+l)]]$$

This is:

- invariant to changes in gain, bias, illumination

- robust to occlusion
- can be parallelized
- only computes on integers (fast and precise)